

Algorithms

Lecture 8:

Applications of Sorting

Dr. Wang, Jingwen
Department of Computer Science

Lecture outline

Announcements/notes

- Team Homework 1
- Daily Problems 8
- Reading ADM page 109-114

Today's lecture

- Daily Problems 7
- Applications of Sorting



Daily Coding Problems 7

*1512. Number of Good Pairs**Links*



Daily Coding Problems 7

705. Design HashSet

Doubles Tennis game

- Suppose these are skills of tennis players.
- For the Doubles Tennis game, pair the players into teams such that it would be fair.

Exercise

- Take as input a sequence of n real numbers.
- Design an algorithm that partitions the numbers into n pairs, with the property that the partition **minimizes the maximum sum of a pair**.
- For example,
 - input numbers (1, 5, 3, 9)
 - the possible partitions are
 - ((1, 3), (5, 9)) $\rightarrow$ sum (4, 14) $\rightarrow$ max sum 14
 - ((1, 5), (3, 9)) $\rightarrow$ sum (6, 12) $\rightarrow$ max sum 12
 - ((1, 9), (3, 5)) $\rightarrow$ sum (10, 8) $\rightarrow$ max sum 8 – smallest max sum
- Thus, the third partition has the smallest maximum sum over all partitions.

Exercise

- Take as input a sequence of n real numbers.
- Design an $O(n \log n)$ algorithm that partitions the numbers into n pairs, with the property that the partition **minimizes the maximum sum of a pair**.
- For example,
 - input numbers (1, 5, 3, 9)
 - the possible partitions are
 - ((1, 3), (5, 9)) $\rightarrow$ sum (4, 14) $\rightarrow$ max sum 14
 - ((1, 5), (3, 9)) $\rightarrow$ sum (6, 12) $\rightarrow$ max sum 12
 - ((1, 9), (3, 5)) $\rightarrow$ sum (10, 8) $\rightarrow$ max sum 8 – smallest max sum
- Thus, the third partition has the smallest maximum sum over all partitions.

Solution

- To pair up numbers to minimize the sum, the largest element in the partition must be added to the smallest element in the array.
- Thus, we can
 - Step1: sort the input numbers, and
 - Step 2: use two pointers to either end of an array, pairing up the largest number with the smallest number, the second-largest number with the second smallest number, and so on.
 - Step 3: the max sum from these pairs is the smallest max sum over all possible partitions.

Sorting

Importance of Sorting

- Why don't CS profs ever stop talking about sorting?
 - Computers spend a lot of time sorting, historically 25% on mainframes.
 - Sorting is the best studied problem in computer science, with many different algorithms known.
 - Most of the interesting ideas we will encounter in the course can be taught in the context of sorting, such as divide-and-conquer, randomized algorithms, and lower bounds.
- You should have seen most of the algorithms, so we will concentrate on the analysis.

Efficiency of Sorting

- Sorting is important because that once a set of items is sorted, many other problems become easy.
- Further, using $O(n \log n)$ sorting algorithms leads naturally to sub-quadratic algorithms for all these problems.

n	n^2	$n \lg n$
10	25	33
100	2,500	664
1,000	250,000	9,965
10,000	25,000,000	132,877
100,000	2,500,000,000	1,660,960
1,000,000	250,000,000,000	13,815,551

Pragmatics of Sorting: Library Functions

- Any reasonable programming language has a built-in sort routine as a library function.
- You are almost always better off using the system sort than writing your own routine.
- For example, the standard library for C contains the function `qsort` for sorting:

```
#include <stdlib.h>

void qsort(void *base, size_t nel, size_t width,
           int (*compare) (const void *, const void *));

int intcompare(int *i, int *j)
{
    if (*i > *j) return (1);
    if (*i < *j) return (-1);
    return (0);
}
```

Sort An Array in Python (1)

Python



```
my_list = [3, 1, 4, 1, 5, 9, 2, 6]
my_list.sort()
print(my_list)  # Output: [1, 1, 2, 3, 4, 5, 6, 9]

my_list.sort(reverse=True)
print(my_list)  # Output: [9, 6, 5, 4, 3, 2, 1, 1]
```

Sort An Array in Python (2)

Python



```
my_list = [3, 1, 4, 1, 5, 9, 2, 6]
new_list = sorted(my_list)
print(new_list)  # Output: [1, 1, 2, 3, 4, 5, 6, 9]
print(my_list)   # Output: [3, 1, 4, 1, 5, 9, 2, 6] (original list remain
```

Sort An Array in Python (3)

Python



```
my_list = ["apple", "banana", "cherry", "date"]  
my_list.sort(key=len)  
print(my_list) # Output: ['date', 'apple', 'banana', 'cherry']
```

Sort An Array in Java (1)

Java



```
import java.util.Arrays;

public class SortArray {
    public static void main(String[] args) {
        int[] numbers = {5, 2, 9, 1, 5, 6};
        Arrays.sort(numbers);
        System.out.println(Arrays.toString(numbers));

        String[] names = {"Charlie", "Alice", "Bob"};
        Arrays.sort(names);
        System.out.println(Arrays.toString(names));
    }
}
```


Sort An Array in Java (2)

Java



```
import java.util.Arrays;

public class SortSubArray {
    public static void main(String[] args) {
        int[] numbers = {5, 2, 9, 1, 5, 6};
        Arrays.sort(numbers, 1, 4); // Sorts from index 1 (inclusive) to
        System.out.println(Arrays.toString(numbers));
    }
}
```

Sort An Array in Java (3)

Java



```
import java.util.Arrays;
import java.util.Collections;

public class SortArrayDescending {
    public static void main(String[] args) {
        Integer[] numbers = {5, 2, 9, 1, 5, 6};
        Arrays.sort(numbers, Collections.reverseOrder());
        System.out.println(Arrays.toString(numbers));
    }
}
```

Sort An Array in C

C



```
#include <stdio.h>
#include <stdlib.h>

int compare_integers(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int numbers[] = {5, 2, 9, 1, 5, 6};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    qsort(numbers, size, sizeof(int), compare_integers);

    printf("Sorted array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");
    return 0;
}
```

Applications of Sorting

Application 1: Searching

- Binary search lets you test whether an item is in a dictionary in $O(\lg n)$ time.
- Search preprocessing is perhaps the single most important application of sorting.

Application 2: Closest pair

- Given n numbers, find the pairs which are closest to each other.
- Once the numbers are sorted in $O(n \lg n)$, the closest pair will be next to each other in sorted order, so an $O(n)$ linear scan completes the job.

Application 3: Element Uniqueness

- Given a set of n items, are they all unique or are there any duplicates?
- Sort them and do a linear scan to check all adjacent pairs.
- This is a special case of closest pair above.

Application 4: Mode

- Given a set of n items, which element occurs the largest number of times? More generally, compute the frequency distribution.
- Sort them and do a linear scan to measure the length of all adjacent runs.

Application 5: Median and Selection

- What is the k th largest item in the set?
- Once the keys are placed in sorted order in an array, the k th largest can be found in constant time by simply looking in the k th position of the array.

Selection Sort / Heapsort

Selection Sort

- Selection sort scans through the entire array, repeatedly finding the smallest remaining element.

```
For  $i = 1$  to  $n$ 
```

```
  A: Find the smallest of the last  $n - i + 1$  items.
```

```
  B: Pull it out of the array and put it first.
```

- Selection sort takes $O(n (T(A) + T(B)))$ time.
- For Array, selection sort takes $O(n^2)$.

The Data Structure Matters

- Using **arrays** or **unsorted linked lists** as the data structure, operation A takes $O(n)$ time and operation B takes $O(1)$, for an $O(n^2)$ selection sort.
- Using **balanced search trees** or **heaps**, both of these operations can be done within $O(\lg n)$ time, for an $O(n \log n)$ selection sort called **heapsort**.
- Balancing the work between the operations achieves a better tradeoff.
- Key question: “Can we use a different data structure?”